Product Selection Optimization

Under Budget Constraints

An NP-Hardness Analysis, Dynamic Programming & Greedy Heuristic Approach

Esteban Murillo · Sebastian Lopez · Diego Bonilla
May 2026 | Automata Theory

Problem Statement

The Scenario

A store manager must select products to maximize promotional value while staying within a fixed budget or shelf-space limit.

Each product has:

- Cost / shelf space used
- Profit / promotional value
- Budget / shelf-space limit

Goal: maximize total value without exceeding budget.

Knapsack → Product Selection

Knapsack	Product Selection
Item	Product
Weight	Cost / shelf space
Value	Profit / promo value
Capacity W	Budget B

NP-Hardness: Reduction from 0–1 Knapsack

1

Start with a known NP-hard problem

The 0–1 Knapsack Problem is NP-hard: given items with weights w_i and values v_i, find a subset maximizing total value subject to total weight $\leq W$.

2

Construct the polynomial transformation

For each knapsack item (w_i, v_i), create one product with cost $c_i = w_i$ and profit v_i. Set budget $B = W$. This mapping runs in $O(n)$ time.

3

Conclude NP-hardness

Any optimal product-selection solver also solves Knapsack optimally — in polynomial time. Since Knapsack is NP-hard, Product Selection is NP-hard too.

∴ Product Selection Under Budget Constraints is NP-hard.

Optimal Solution: Bottom-Up Dynamic Programming

Algorithm Design

Build table dp[i][w] = max value using first i products with budget w:

dp[i][w] =

```
dp[i-1][w]                          if c_i > w
max(dp[i-1][w], v_i + dp[i-1][w-c_i])  if c_i ≤ w
```

Backtracking recovers the optimal subset in O(n).
Optimal value = dp[n][W].

Time Complexity

$O(n \cdot W)$ — pseudopolynomial
Depends on numeric value of W, not just input length.

Test Datasets

n	Budget W	n·W
5	20	**100**
10	50	**500**
20	100	**2000**
30	200	**6000**
50	300	**15000**

All cases use costs 1…n, values 10…10n

Greedy Heuristic: Value-to-Weight Ratio

1

Compute ratios

For each product i, calculate profit[i] / cost[i]

2

Sort descending

Order items from highest to lowest ratio — O(n log n)

3

Greedily select

Include each item if it fits within remaining budget

4

Return result

Total profit and selected indices — O(n) space

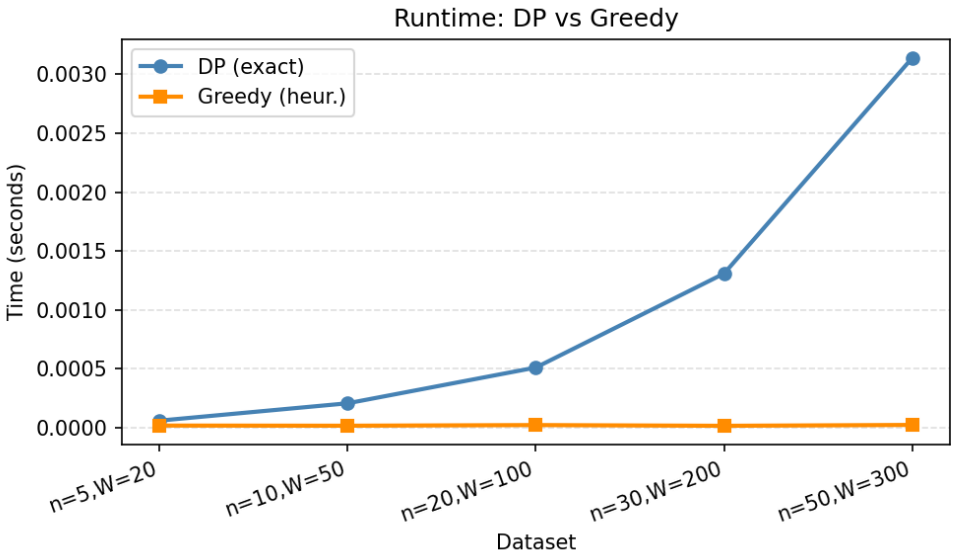
Complexity

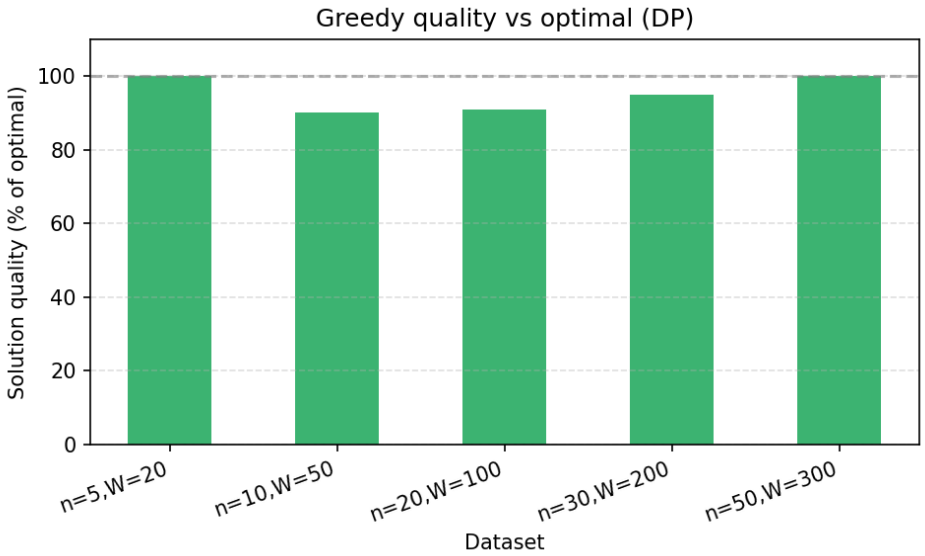
O(n log n) time · O(n) space
Far faster than DP for large n and W

Trade-off

Not guaranteed optimal — but achieves near-100% solution quality on structured datasets

Empirical Results: Runtime Comparison

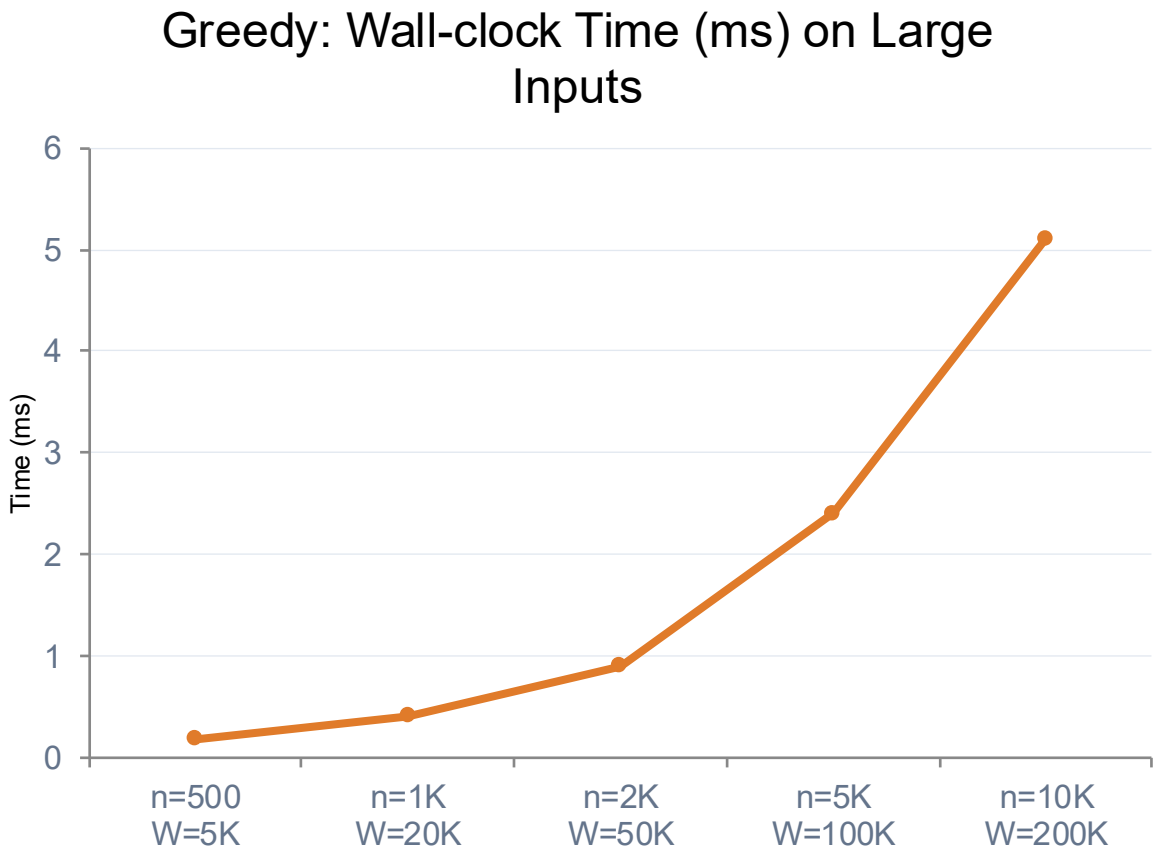




- The blue line is the DP, it grows noticeably as n and W increase.
- The orange line is the greedy, it's basically flat, hugging the bottom the whole time.
- On the largest base dataset, n equals 50 and W equals 300, the DP takes about 3 milliseconds while greedy finishes in under 0.03 milliseconds, that's roughly 130 times faster.

- The greedy hits 100% on two datasets, and on the other three it gets 90, 91, and 95 percent of optimal.
- So you're giving up almost nothing in quality but getting a massive speedup.

Greedy Scalability: Large-Scale Experiments



O(n log n)

Greedy complexity — scales gracefully

~5ms greedy

Time for n=10,000 products, W=200,000, DP would need a 2 billion cells table at that scale, not feasible.

100%

Solution quality on all base datasets

Algorithm Comparison: DP vs Greedy

	DP (Exact)	Greedy (Heuristic)
Optimality	✓ Guaranteed	✗ Not guaranteed
Time complexity	O(n · W) pseudo-poly	O(n log n) polynomial
Space	O(n · W)	O(n)
Best for	Small n, small W	Large n or W
Scales to n=10,000?	✗ Slow	✓ ~5 ms
Quality (test sets)	100% (optimal)	100% of DP optimal